TECHNICAL REFERENCE & MANUAL

Website Architecture & Component Anatomy

A comprehensive breakdown of Varun Patel's iOS Developer Portfolio

1. What Makes a Web Page? The Core Triad

Every interactive website on the internet is created using three primary code technologies working together in harmony. Understanding how these layers separate responsibilities is key to clean website development:

1. HTML (Structure)

HyperText Markup Language defines the semantic content, text, headings, images, and structural hierarchy of the website (the "skeleton").

2. CSS (Styling)

Cascading Style Sheets controls colors, typography, margins, layouts (Flexbox/Grid), dark mode, and visual aesthetics (the "skin & styling").

3. JavaScript (Behavior)

JavaScript provides client-side interactivity, event listeners, dynamic filtering, state changes, and DOM manipulation (the "brain").

4. Document Object Model (DOM)

The **DOM** is the live tree structure that JavaScript modifies to update the screen in real-time without refreshing the page.

2. Page Document Structural Anatomy

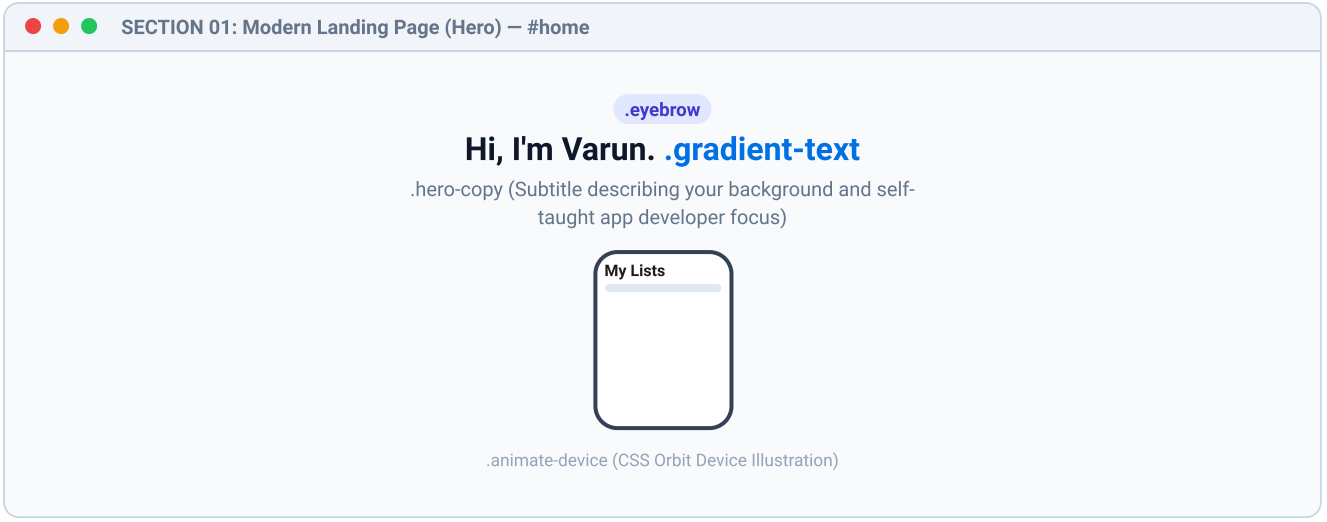
A standard web page follows an industry-standard layout pattern. Below is the structural hierarchy used in your portfolio:

HTML Element	Standard Class / ID Name	Architectural Purpose
`<header>`	`.nav-wrap` / `.glass-nav`	Top sticky navigation bar containing brand logo, page links, & CTA buttons.
`<main>`	`main`	Container wrapping the core content sections (Hero, Story, Tools, Projects).
`<section>`	`#home`, `#about`, `#projects`	Independent thematic content blocks grouped logically with unique IDs.
`<footer>`	`footer`	Bottom site bar holding copyright, legal disclaimers, and jump links.

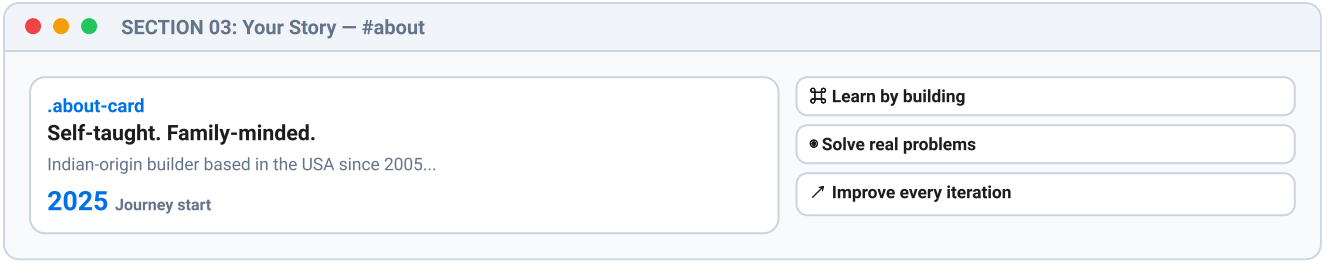
Self-Learning Tip: Always use semantic tags like `<header>`, `<main>`, and `<section>` instead of generic `<div>` tags. This improves accessibility for screen readers and boosts search engine rankings (SEO).

3. Detailed Breakdown: Sections 01 & 03

Below is a visual and technical breakdown of the top two sections on your site, detailing exact class naming conventions and development key points.

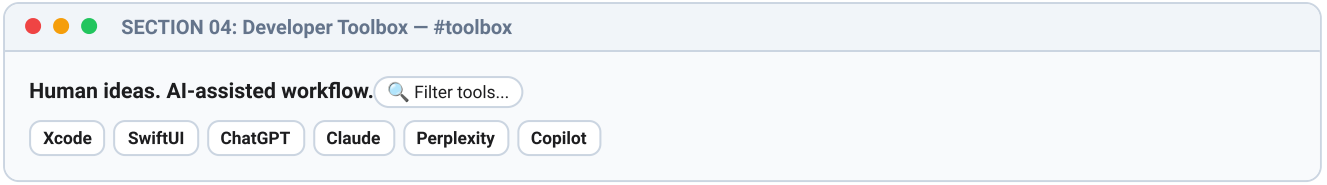


Component Element	Class / Identifier	Development Key Points & Rules
Hero Container	`header.hero` / `#home`	Uses flexbox centering and high responsive vertical padding (`pt-16` `pb-16`).
Gradient Text	`.gradient-text`	Uses `-webkit-background-clip: text` with a multi-stop color linear gradient.
CSS Device Mockup	`.animate-device`	Pure CSS mobile mockup rendered without heavy image files for lightning-fast loads.

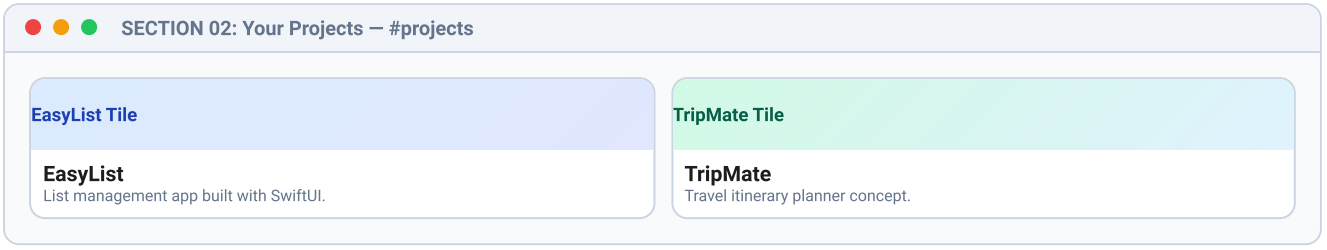


4. Detailed Breakdown: Sections 04 & 02

Analyzing the developer toolbox and project grid sections, focusing on dynamic interactive elements and CSS Grid layouts.



Toolbox Component	Class / Identifier	Development Key Points
Live Search Input	`#toolSearch`	Connected to JavaScript event listener to dynamically hide/show matching chips.
Tool Chips	`.tool-chip`	Interactive cards styled with subtle hover border transforms (`hover:border-blue-500`).



Project Component	Class / Identifier	Development Key Points
Filter Category Bar	`.project-filter-btn`	Uses `data-filter` attributes ("productivity", "travel") to isolate cards.
Project Cards	`.project-card`	2-column desktop grid collapsing cleanly to 1 column on mobile phones (`grid-cols-1 md:grid-cols-2`).

5. Section 05: Responsive Features & Modal Architecture

Section 05 encompasses the interactive contact panel, automatic dark mode setup, and modal popups. Understanding these utilities will help you edit site behavior safely.

Feature Name	Code Implementation	How It Works
Automatic Dark Mode	`@media (prefers-color-scheme: dark)`	Detects user OS dark mode settings automatically without needing a manual toggle button.
Interactive Modal	`#projectModal`	Fixed backdrop dialog overlay populated dynamically using JavaScript functions (`openModal()`).
Email Copy Toast	`copyEmail()` & `#toast`	Copies text to clipboard using Document commands and displays a temporary notification popup.

6. Recommended Naming Conventions for Edits

When making future modifications or adding new features to your codebase, strictly follow these web development naming standards:

Kebab-Case for CSS Classes

Always use lowercase separated by hyphens (e.g., `.project-card` , `.glass-nav`). Avoid camelCase in HTML/CSS.

camelCase for JavaScript

Use camelCase for JS variables and functions (e.g., `copyEmail()` , `openModal()` , `toolSearch`).

Unique IDs for JS Anchors

Use IDs strictly for unique section targets (`#home` , `#about`) or specific JS hooks (`#mobileMenu`).

Semantic Data Attributes

Use `data-*` attributes (e.g. `data-category="travel"`) to store custom metadata for filtering.

Self-Learning Summary: A professional website is a modular collection of well-named, structured components. By separating HTML structure, CSS layout tokens, and JS event handling, you ensure your code remains scalable, clean, and easy to maintain as your developer portfolio expands!